

SRMIC

SRAM-Residency Memory-Centric Inference Chip Architecture Whitepaper - detailed conceptual design study

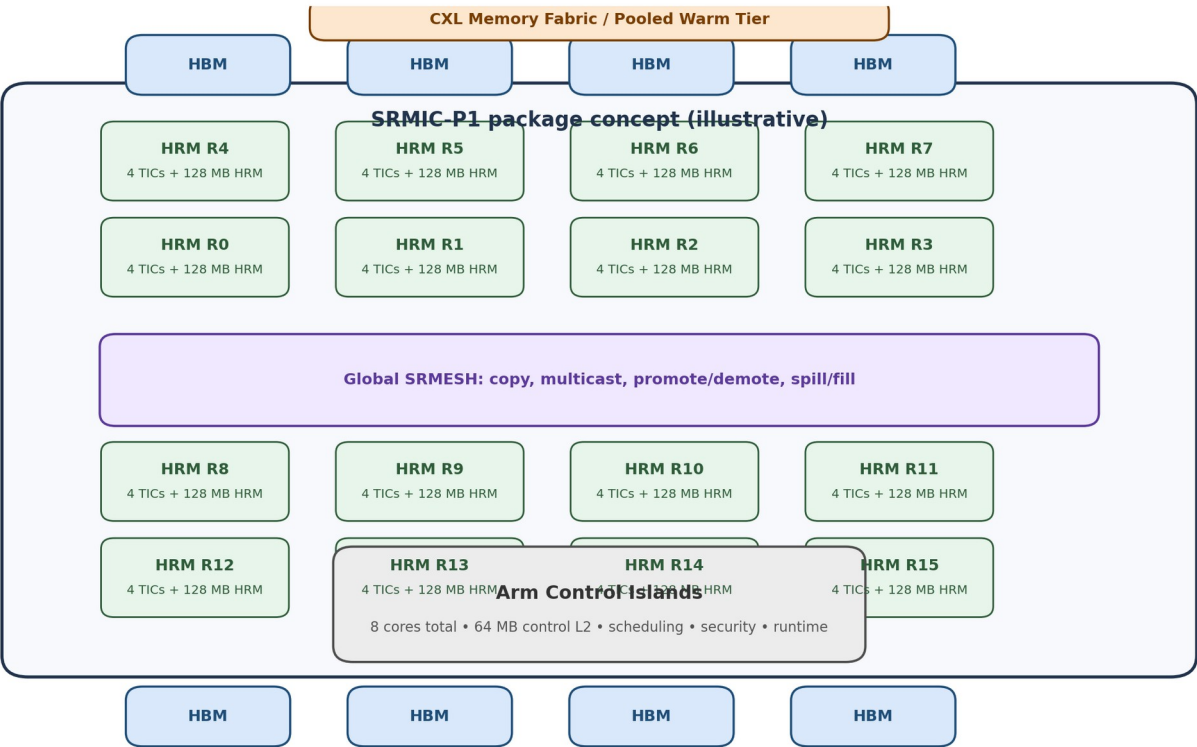


Figure 1. Illustrative SRMIC-P1 package architecture with HRM, HBM, SRMESH, Arm control islands, and CXL pooled warm tier.

Document intent

This document is not a sign-off implementation package. It is a serious architecture memo intended to be detailed enough for concept review, investor discussion, invention framing, and early feasibility analysis. Quantitative values are first-order engineering assumptions selected to make the design space explicit and internally consistent.

Contents

1. 1. Executive summary
2. 2. Design assumptions and engineering target
3. 3. Why a new inference architecture is needed
4. 4. SRMIC design goals and non-goals
5. 5. System overview
6. 6. Package architecture
7. 7. HRM region microarchitecture
8. 8. Tensor inference cluster microarchitecture
9. 9. Arm control plane and software services
10. 10. Memory hierarchy and residency semantics
11. 11. SRMESH interconnect design
12. 12. HBM subsystem and cold-tier policy
13. 13. CXL warm-tier expansion and pooling
14. 14. Capacity sizing model
15. 15. Bandwidth model
16. 16. Token latency model
17. 17. Area allocation model
18. 18. Power and thermal model
19. 19. Packaging and physical implementation options
20. 20. Node-level scale-out architecture
21. 21. Competitive positioning
22. 22. Risks, open questions, and mitigation plan
23. 23. Suggested development roadmap
24. 24. Invention / patent framing
25. 25. Appendix A - consolidated assumptions
26. 26. Appendix B - reference notes
27. 27. Appendix C - page-state metadata and movement state machine
28. 28. Appendix D - compiler, runtime, and software contract
29. 29. Appendix E - QoS, isolation, and multi-tenant controls
30. 30. Appendix F - reliability, serviceability, and observability
31. 31. Appendix G - worked workload scenarios
32. 32. Appendix H - sensitivity and what must be true for SRMIC to win
33. 33. Appendix I - product stack and packaging roadmap
34. 34. Appendix J - due-diligence questions for the next review
35. 35. Appendix K - glossary and design shorthand
36. 36. Appendix L - design review checklist and FAQ

1. Executive summary

SRMIC is an inference-first accelerator architecture built around the idea that the dominant cost in modern large-model inference is the movement of bytes, not the arithmetic itself. In a conventional GPU path, the inference engine repeatedly traverses a long ladder: tensor core <-> local register / shared storage <-> cache hierarchy <-> HBM. That path works extremely well for broad programmability and training-scale throughput, but it is not inherently optimized for deterministic, low-jitter, low-batch token generation when the active working set is repeatedly reused across adjacent decode steps.

The SRMIC answer is to promote a different design object to first-class status: a very large hot-residency SRAM tier, called HRM, physically placed near the execution clusters and treated as an explicitly managed inference substrate rather than as a passive cache. The package then adds HBM as a cold / warm package-local tier and CXL as a pooled warm-tier expansion fabric. Arm cores remain present, but only as a control plane: scheduling, residency management, telemetry, protection, and host-facing software services. The chip is not CPU-centric; it is memory-centric.

This document chooses a concrete but ambitious package as the anchor point for analysis. The baseline package, called SRMIC-P1, uses four logic chiplets inside one package, with a package-level aggregate of 4 GB HRM, 8 HBM stacks, 8 Arm control cores, and 128 tensor inference clusters. A four-package node, SRMIC-N1, then scales the concept to 16 GB aggregate HRM and 512 tensor clusters. These values are illustrative, not sacred; the purpose is to make trade-offs legible.

The central claim of the architecture is not that SRAM magically replaces HBM in all roles. It does not. Instead, SRMIC argues for a tiered execution model: keep the token-critical path inside HRM plus nearby tensor clusters; use HBM for colder pages, long-tail weights, and overflow state; use CXL for pooled, less latency-sensitive warm capacity; and manage the transitions explicitly rather than hoping a generic cache replacement policy will do the right thing.

The rest of the paper develops that claim from multiple angles: package organization, HRM region structure, compute microarchitecture, memory semantics, interconnect topology, sizing equations, latency math, bandwidth envelopes, area splits, power and thermal implications, packaging choices, node-level scale-out, competitive comparison, and a candid risk register.

Short thesis

SRMIC is best thought of as an Arm-orchestrated, SRAM-residency inference fabric with HBM as a cold tier and CXL as a pooled warm tier. It is neither a CPU with oversized cache nor a generic GPU with a slightly larger LLC.

SRMIC memory hierarchy

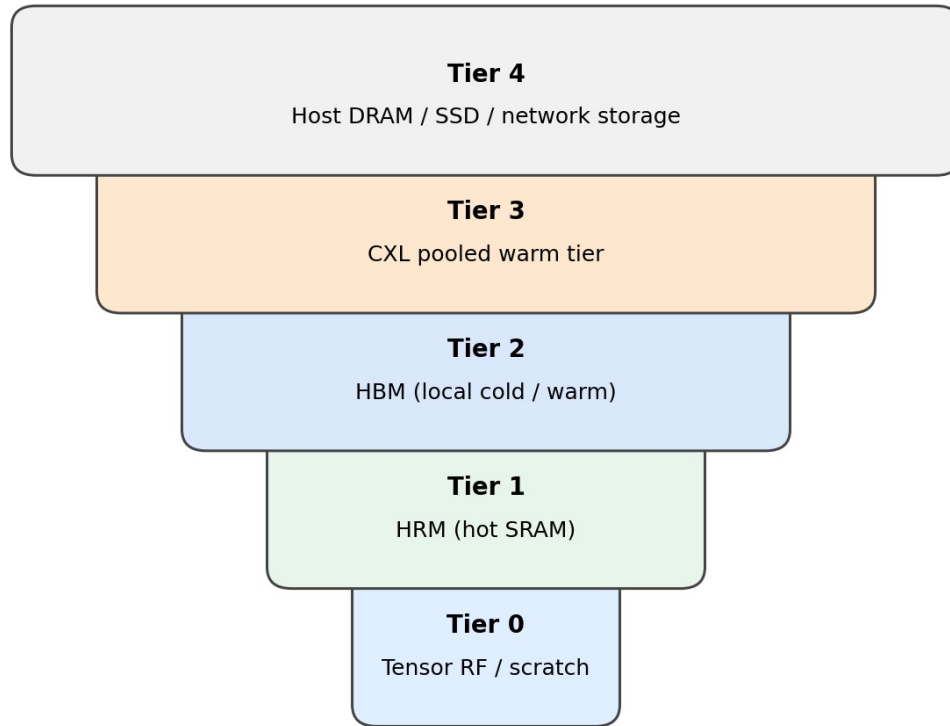


Figure 2. The intended SRMIC memory hierarchy. Tier 1 HRM is the primary execution substrate for the hot path; HBM and CXL sit behind it.

2. Design assumptions and engineering target

This study picks assumptions that are aggressive enough to be interesting yet bounded enough to remain discussable as a near-to-mid-term product concept. The logic process assumption is an advanced leading-edge node in the 3 nm class for the primary logic and SRAM macros, combined with 2.5D package integration for HBM and a chiplet-based logic floorplan. More speculative variants - for example, 3D stacked SRAM tiers above logic - are discussed later, but they are treated as roadmap options, not baseline implementation commitments.

The workload assumption is decoder-heavy generative inference, with emphasis on low-batch and medium-batch serving for models that exhibit strong layer-to-layer reuse of weights and active key-value state. The architecture is intended to support modern transformer-style large language models, multimodal decoders, and bounded-context enterprise copilots. It is not optimized first for dense training, broad CUDA-style programmability, or extremely latency-insensitive throughput jobs.

The package power target is intentionally high. In order to keep the analysis honest about what large SRAM plus substantial tensor throughput implies, the baseline package is treated as an approximately 800-1000 W class

accelerator package. That class is ambitious but not absurd for a premium data-center inference part. It allows the paper to discuss thermal pressure directly rather than hiding it.

The product positioning assumption is a data-center inference processor, not an edge SoC. Edge devices are constrained by much smaller power and package budgets and would push the architecture toward a different optimum. SRMIC as described here lives in the low-latency server / appliance / accelerator-card regime, where package complexity is acceptable if it materially improves cost per useful token or service-level latency under interactive load.

The package decomposition assumed throughout this paper is shown in Table 1. Readers should treat it as the default anchor, not as a final tapeout claim.

Table 1. Baseline illustrative package assumptions	
Logic organization	4 logic chiplets in one package, package-level coherent control plus explicit residency fabric
Control plane	8 Arm cores total, organized as 4 control islands with regional L2
Hot tier	4 GB total HRM SRAM, physically distributed across 32 regions
Execution plane	128 tensor inference clusters, 4 TICs per HRM region
Cold tier	8 HBM stacks used for colder weights, overflow KV, and page staging
Warm expansion	CXL-attached pooled memory / expansion modules
Target regime	Interactive and enterprise LLM inference, low jitter, bounded hot-set reuse

Interpretation guide

Whenever the paper uses terms such as 'faster', 'better', or 'lower latency', read them as relative to an HBM-dominant inference path under the same broad performance class and under the explicit assumption that the active working set hit rate in HRM is sufficiently high. SRMIC is not a universal replacement for all accelerators.

3. Why a new inference architecture is needed

Modern large-model serving has pushed hardware into an awkward place. The arithmetic engines are extraordinarily capable, but the inference experience that matters to many customers - first token latency, steady-state token latency, jitter under concurrency, and cost per interactive session - often depends more on whether the active bytes are already close to compute than on peak tera-operations. The gap between arithmetic throughput and the cost of moving data has therefore become the architecture problem.

HBM is an enormous achievement and remains indispensable. It offers very high bandwidth and practical capacity density, especially compared with SRAM. But HBM still sits behind PHYs, controllers, package wiring, and longer access paths than local SRAM. That is acceptable and often optimal for broad workloads. It is not obviously optimal for every decode step of every user-facing inference transaction, especially if many of the same pages are reused within narrow time windows.

A second issue is that a general-purpose cache hierarchy is usually not aware of semantic hotness in the way inference software is. It knows access recency and maybe a few hints, but it usually does not know that a

particular expert, prompt prefix, active KV band, or quantized weight page is likely to be needed in the next few decode windows of the same or adjacent sessions. When the semantic knowledge is outside the hardware, the architecture often over-pays in bandwidth and latency.

SRMIC therefore starts from a different premise: inference can be treated as a residency problem. If the hardware and runtime jointly know which pages are truly hot, which pages are likely next, and which pages can be demoted without hurting service quality, then the memory hierarchy can be shaped around that knowledge. SRAM becomes the expensive but very effective hot substrate. HBM becomes a crucial but colder tier. CXL becomes a pooled warm backstop. The execution engine is placed directly in that hierarchy.

- Primary bottleneck for interactive inference is frequently data placement, not raw compute peak.
- Hot working sets are often smaller than full model size and therefore economically worth treating differently.
- Generic cache replacement is too weak a semantic signal for modern serving workloads.
- Determinism matters: users experience jitter, not peak FLOPS.
- Architectures that shorten the active path can win even without dominating on total package bandwidth.

4. SRMIC design goals and non-goals

The architecture is easier to evaluate once its goals are written plainly.

Goal	Meaning in practice
Low decode jitter	Token path should avoid unnecessary excursions into colder tiers once active pages are resident.
Explicit hot-set control	Promote, pin, multicast, demote, and spill are hardware-visible semantics, not only software conventions.
Package-level composability	A single package should be useful; a four-package node should scale without rewriting the mental model.
Realistic ecosystem anchor	Arm control plane, HBM cold tier, and CXL warm tier use established industry ecosystems where possible.
Competitive manufacturability	The concept should avoid requiring wafer-scale silicon as the baseline implementation path.

Non-goal	Reason
Best training accelerator	Training imposes different memory patterns, optimizer state behavior, and software expectations.
Maximal generic programmability	The design intentionally specializes around residency-managed inference.
Pure SRAM-only execution for huge models	Capacity economics make a pure-SRAM strategy weak beyond the hot tier.
Phone / edge power envelope	The current concept is aimed at premium data-center deployment, not tens-of-watts systems.

Design posture

This architecture is deliberately narrow enough to be differentiated. Broad programmability is valuable, but it is not the first design center here. The first design center is useful interactive inference.

5. System overview

At the package level, SRMIC contains three architectural planes that must be understood together. The first is the control plane: Arm control islands with regional caches, firmware services, telemetry, scheduling logic, protection management, and host interfaces. The second is the execution plane: tensor inference clusters, abbreviated TICs, placed near HRM slices and specialized for the kernels that dominate modern inference. The third is the residency plane: HRM itself, plus the fabric logic that moves pages among HRM, HBM, and CXL.

The entire package is wrapped by HBM stacks around the logic footprint and is attached upward to a CXL memory fabric for pooled expansion. This means the architecture is neither monolithic SRAM worship nor HBM denial. It is a multi-tier design in which each tier is assigned a specific latency and policy role.

Another important aspect of the overview is that the package avoids turning the Arm cluster into the center of data movement. Arm is present to direct, account for, and secure the system, but the package should not force the CPU to manually shepherd page-by-page traffic at inference rate. The fast path must be expressed in descriptors and residency policies that the package executes mostly in hardware.

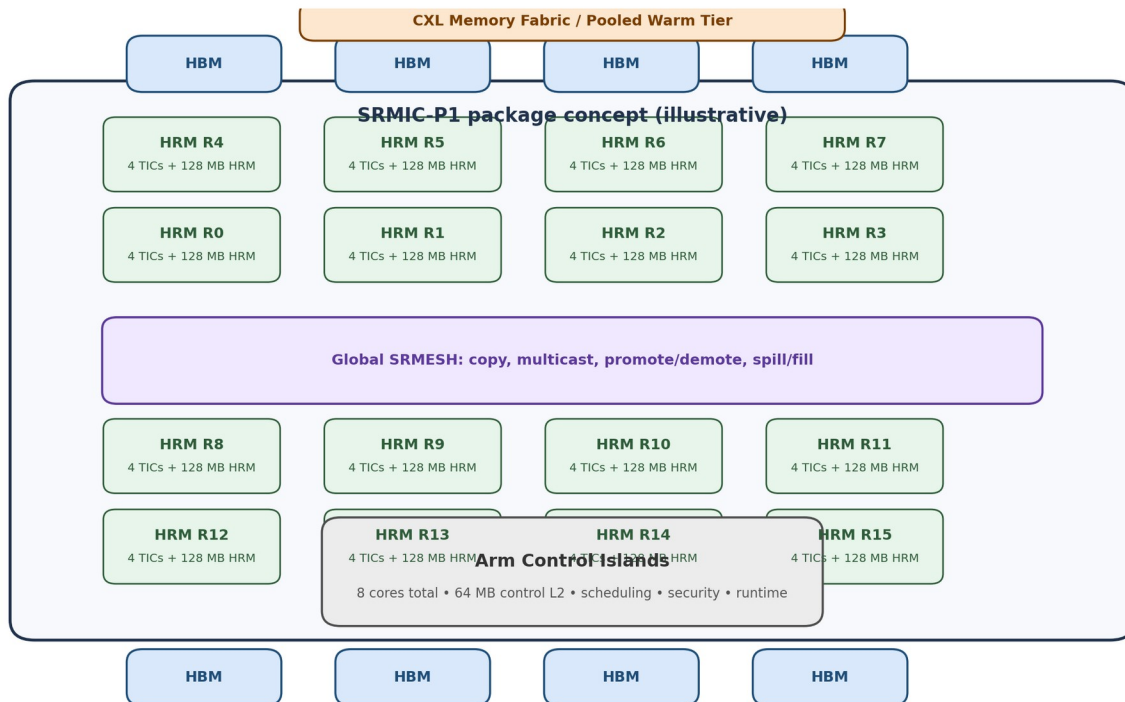


Figure 3. Package-level overview. Each HRM region combines SRAM banks, tensor clusters, and a local residency controller; the package ties regions together with SRMESH.

6. Package architecture

The baseline package uses four logic chiplets, each contributing a share of the total control logic, HRM, and tensor execution resources. This choice is motivated by practical physical-design concerns. A single enormous die dominated by SRAM and wide global interconnect quickly runs into reticle, yield, routing, and power-delivery pain. A multi-chiplet package is not free, but it is often a more believable way to reach multi-gigabyte hot SRAM capacity than a single giant reticle-limited die.

Within the package, each chiplet contains eight HRM regions. Those regions are arranged so that local region traffic remains physically short, and package-level traffic only occurs when data must move between regions on

different chiplets or when pages are promoted from HBM into the local hot tier. The package-level SRMESH is therefore hierarchical: strong locality by default, wider movement only when required.

HBM stacks are positioned around the package periphery. The package should ideally maintain balanced proximity from each logic chiplet to at least two HBM stacks so that cold-tier traffic does not create a single lopsided pressure point. The CXL-facing logic is attached through the control-plane side of the package, because warm-tier traffic is more policy-rich and less strictly in the immediate inner decode loop than HRM or HBM traffic.

Parameter	Illustrative baseline	Architectural rationale
Chiplets per package	4	Spreads SRAM and routing load; keeps single-die implementation pressure manageable
HRM regions per chiplet	8	Allows a regular floorplan and bounded region-local hop count
TICs per region	4	Enough local compute to exploit region-local hot pages without over-concentrating power
HBM stacks per package	8	Maintains strong local cold-tier bandwidth and practical package density
Control islands per package	4	Maps naturally onto 4 chiplets and regional scheduling domains

7. HRM region microarchitecture

The HRM region is the fundamental building block of SRMIC. If the package is the body, the region is the organ. Each region couples four SRAM banks, a local region fabric, four tensor inference clusters, and a residency interconnect controller. The region is intentionally complete enough that meaningful work can often start and end within it.

Treating the region as the primitive gives the architecture three benefits. First, it constrains wire lengths for the most frequent data motions. Second, it gives the runtime a natural placement granularity for hot pages and session state. Third, it makes scale-out from one package to four packages conceptually cleaner, because the hierarchy is already region-based and not purely monolithic.

Each region in the baseline model contains 128 MB HRM split across four 32 MB banks. The split could be adjusted, but the banking itself is non-negotiable. A giant monolithic SRAM block would be physically awkward, harder to clock and power, and worse for parallelism. Banking allows simultaneous reads and writes, local page movement, and multicast source behavior that better matches inference traffic patterns.

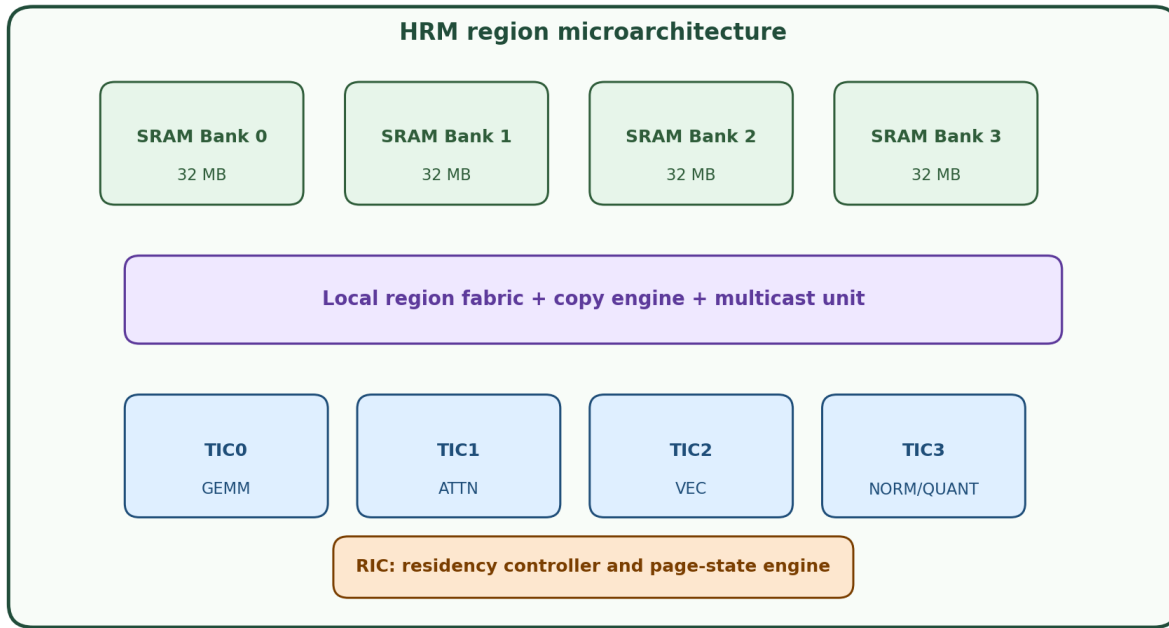


Figure 4. HRM region microarchitecture. In the baseline model each region contains 128 MB HRM and four specialized tensor inference clusters.

- Banking enables parallel accesses, independent low-level power gating, and more graceful physical placement.
- Local copy and multicast hardware prevents the Arm control plane from becoming a byte shepherd.
- Region-local residency control keeps the package from pretending that all hot memory is globally uniform.

Practical reading

A reader should imagine the HRM region less as a cache slice and more as a mini inference island: memory, compute, local control, and page-state logic bound together.

8. Tensor inference cluster microarchitecture

The tensor inference cluster, or TIC, is deliberately narrower in scope than a general GPU streaming multiprocessor. The intent is not to reproduce every dynamic execution pattern under the sun, but to accelerate the kernels that dominate the target inference regime. A reasonable baseline TIC therefore contains a matrix engine for GEMM-like operations, an attention-oriented datapath for KV-heavy operations, a vector lane for elementwise and reduction work, and a fused normalization / quantization support block.

The architecture does not require each TIC to be identical, but the baseline assumes modest specialization: one cluster may be slightly stronger on GEMM, another on attention movement, another on vector and normalization stages. This gives the region flexibility without forcing every local compute tile to pay for every possible functional sub-block.

Each TIC also carries a small local scratch structure that sits below HRM in the hierarchy. This scratch is not the architectural star. It is present to avoid trivial overuse of HRM for tiny transient values and to allow tight fused kernels, but the real hot-state story remains the regional SRAM. That choice is important because it keeps the mental model simple: scratch is immediate, HRM is hot, HBM is cold, CXL is warm pooled.

TIC block	Primary role	Why it exists in SRMIC
Matrix engine	Dense GEMM / projection work	Still dominates many inference steps even when memory is the real bottleneck
Attention datapath	KV gather, weighted accumulation, layout motion	Supports the decode path that is often sensitive to data placement
Vector / reduction lane	Elementwise ops, reductions, activation functions	Reduces dependence on shipping small post-processing steps elsewhere
Norm / quant support	Layernorm, RMSNorm, quant / dequant assists	Keeps locality strong and avoids wasting SRMESH bandwidth on low-byte transforms
Local scratch	Sub-HRM transient storage	Prevents pathological traffic of tiny temporaries into the main regional SRAM

9. Arm control plane and software services

Arm is chosen in SRMIC not because CPUs are the main source of throughput, but because a modern inference package still needs a disciplined orchestration plane. Requests arrive, are admitted, classified, and metered. Model pages are loaded, warm-state is shared, security domains are maintained, and software stacks need a sane programming environment. A control plane with mature ecosystem support matters.

The architecture therefore uses eight Arm cores organized as four control islands. Each island manages the local chiplet neighborhood, including local telemetry, model page accounting, page-state descriptors, thermal coordination, and request scheduling. Control islands can collaborate, but the design should avoid forcing every decision to a single global controller. That would reduce robustness and would create a needless hot spot.

A large regional control L2 is more useful here than a giant CPU-centric shared L3. The control plane works on descriptors, residency metadata, queues, and policy tables. It does not need to behave like the center of tensor math. Therefore the best use of scarce on-package SRAM is usually to allocate most of it to HRM, while giving the control islands enough L2 to keep their own metadata and software path smooth.

Control-plane service	Purpose
Admission and queueing	Accept requests, classify service tier, assign priority and page budget
Residency management	Track page heat, issue promote/demote hints, pin required pages
Telemetry and accounting	Collect misses, region pressure, HBM usage, and token-latency observability
Security and isolation	Maintain tenant partitioning, page ownership, and management-plane safety
Host / CXL coordination	Integrate package-local execution with pooled warm-memory expansion

Why not x86 inside the package?

For this architecture, the CPU is primarily a control-plane instrument. Arm is attractive because it offers a strong ecosystem for custom SoCs and meshes while allowing the package to remain CPU-light and acceleration-heavy. x86 can still be the host platform at the system level.

10. Memory hierarchy and residency semantics

The single most important conceptual shift in SRMIC is that Tier 1 HRM is not treated as a traditional cache that merely hopes to catch what software touches. Instead, the architecture assigns explicit semantics to page movement. A page can be promoted, pinned, demoted, multicast, spilled, or staged. The runtime can provide hints and descriptors; the hardware can track counters and make bounded local decisions; and the system can expose observability for tuning. That is what residency-first means in practice.

The hierarchy is therefore defined by both latency and policy role. Tier 0 is local scratch and register-like storage. Tier 1 HRM is the fast execution substrate. Tier 2 HBM is package-local cold and warm capacity. Tier 3 CXL memory is pooled warm capacity that is slower than HBM but valuable for expansion and sharing. Tier 4 host memory and storage remain outside the serving fast path.

Pages in SRMIC need metadata. At minimum, a practical page-state record includes ownership, format, quantization state, last-touch epoch, predicted next-use class, whether the page is safe to demote, whether it is shared across sessions, and whether it is currently pinned due to service guarantees or in-flight decode work. The hardware does not need perfect omniscience, but it needs enough structure that movement is not guesswork.

Tier	Technology	Policy role	Examples of what lives there
0	Scratch / RF	Immediate local compute staging	Temporary activations, fused-kernel intermediates
1	HRM SRAM	Hot execution tier	Active weights, current KV bands, dequantized hot blocks
2	HBM	Package-local cold / warm tier	Cold weights, overflow KV, staged pages awaiting promotion
3	CXL memory	Pooled warm tier	Prompt caches, less active sessions, adapter pools, sharable warm shards
4	Host memory / storage	Bulk source tier	Model files, checkpoints, cold archives

- Promotion: move page from HBM or CXL into HRM because expected next-use value is high.
- Pinning: keep a page in HRM for a bounded interval or policy epoch despite general pressure.
- Demotion: move a page from HRM to HBM or CXL because active value has fallen.
- Multicast: distribute a page or block once to multiple consumers rather than rereading from a colder tier.
- Spill / fill: explicit movement between tiers under pressure or prefetch planning.

11. SRMESH interconnect design

SRMESH is the on-package fabric that ties the HRM regions together. Its job is not to mimic a CPU coherence network exactly, nor to imitate a full GPU on-chip network without modification. Instead, it is a residency-aware movement fabric optimized for wide, predictable transfers among memory-centric regions. The baseline assumption is a hierarchical design: short local movement within a region, then package-level movement through a mesh that supports cut-through routing, multicast, and bounded congestion management.

A naive architecture would treat every SRAM bank as though it simply hangs from a central controller. That is the wrong shape for SRMIC. The important operations are peer oriented. A region may need to send hot partials to a neighboring region, a shared page may need one-to-many fan-out, or an active band of KV state may need

migration because region pressure has changed. SRMESH should make such operations natural rather than exceptional.

From an implementation perspective, SRMESH should be wide, but width alone is not enough. The fabric must understand movement classes. Low-value best-effort telemetry traffic should not block promotion of an active page required by the next token. Therefore a useful design includes multiple service classes or virtual channels, explicit flow-control accounting, and simple but predictable prioritization.

SRMESH feature	Why it matters
Cut-through forwarding	Reduces hop latency for larger page and partial-result transfers
Multicast support	Avoids repeated reads when one block feeds many TICs or regions
Virtual channels / classes	Separates token-critical movement from telemetry or background prefetch
Bounded retries and credits	Makes congestion visible and controllable rather than chaotic
Region-local crossbars beneath global mesh	Keeps local traffic efficient while preserving package-scale composability

12. HBM subsystem and cold-tier policy

HBM remains essential in SRMIC. Its role is not diminished because SRAM is emphasized; rather, it is clarified. HBM is the package-local capacity tier that makes the architecture practical for useful model sizes. Without HBM, the package would either need implausibly large SRAM or would be limited to very small active footprints. With HBM, SRMIC can keep hot pages in HRM while relying on a robust local store for everything colder.

The policy lesson is that HBM should not be thought of as the preferred path for every active decode event. Instead, HBM acts as the closest colder tier that can stage pages into HRM at high bandwidth. When the hit rate in HRM is high, HBM sees primarily controlled staging traffic, demotions, and miss handling - not every token's dominant path. That distinction is what gives SRMIC a chance to lower latency despite not defeating HBM on raw capacity economics.

In practice, the architecture should support two broad HBM policy modes. One is reactive: handle misses and demand-driven promotion. The other is predictive: prefetch pages from HBM into target regions before the compute demand lands. Predictive mode is where the control plane and residency engine can earn their keep.

HBM traffic class	Examples
Reactive fills	Unexpected page needed for the next decode step
Predictive prefetch	Likely next expert or next layer block loaded ahead of time
Demotions from HRM	Hot page cooled off but is still worth retaining package-locally
Overflow KV writes	State no longer worth HRM residency but still package-locally useful
Bulk model initialization	First load of model pages before steady-state serving begins

13. CXL warm-tier expansion and pooling

CXL is one of the most useful enablers for SRMIC because it lets the architecture scale capacity without pretending that every useful byte must be on package. The correct role for CXL in this design is not token-critical memory. It is pooled warm memory: session state that is less active, prompt caches, adapters, sharable model shards, and warm pages that are worth retaining but not worth occupying scarce HBM or HRM.

This distinction matters because it prevents the design from becoming magical thinking. If the package were to assume that CXL sits directly in the innermost latency loop, it would be vulnerable to the very kind of interconnect and software overhead that SRMIC is trying to reduce. But if CXL is instead used as a pooled warm tier, it becomes extremely attractive. It enlarges the system working set, supports multi-package reuse, and reduces duplication of warm assets across nodes.

A practical CXL deployment for SRMIC may include dedicated memory expansion modules that themselves carry memory-controller intelligence, compression support, or page accounting. The main point is that the SRMIC package should see CXL as a manageable external residency tier, not as a transparent replacement for HRM or HBM.

- CXL is best used for warm pooled capacity, not for the critical per-token inner loop.
- CXL helps when many sessions share reusable state that would be wasteful to duplicate inside each package.
- CXL becomes more valuable as the deployment moves from one package to several packages or to a rack appliance.

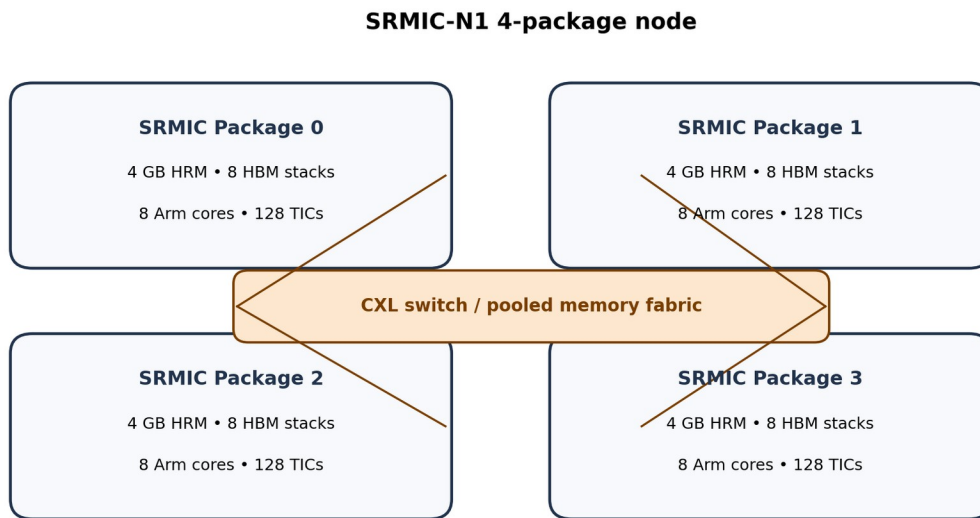


Figure 5. Four-package node concept. CXL connects packages to pooled warm memory and shared service-level state.

14. Capacity sizing model

A serious architecture memo should show its math, even if the math is only first-order. The key sizing question is not 'how big is the full model?' but 'how much of the active working set must be resident at Tier 1 to materially change latency and jitter?' In other words, the design target is not full model residency. It is economically useful hot-set residency.

Consider a decoder serving regime in which a small set of layer blocks, active KV windows, and session-local prompt fragments dominate the next several decode steps. Let H denote the hot-set bytes that truly benefit from remaining in HRM during a scheduling epoch. Let B denote the package-local cold-set bytes best kept in HBM. Let W denote the pooled warm-set bytes that are not latency-critical enough for HBM but are still too valuable to discard. SRMIC is compelling only if H is large enough to matter and small enough to be economically provisioned in SRAM.

The baseline package assumes 4 GB HRM. This does not imply that every deployment needs exactly 4 GB of hot-state. It does, however, create a useful analysis anchor. For example, if we reserved roughly half of HRM for hot model pages and half for active KV / prompt / transient hot state, the package would still have enough room for a meaningful hot-set story without claiming impossible full-model residency.

A practical analysis therefore evaluates several scenarios: a compact 7B-class service where a large fraction of the true hot set can fit; a 13B to 34B-class service where HRM stores the most frequently reused hot layers and current KV windows; and larger services where HRM acts as a high-value residency accelerator rather than a majority-capacity tier.

Scenario	Illustrative role of 4 GB HRM	What that means
Compact model service	Large fraction of hot weights and active KV can live in HRM	Strong chance of very high hit rates and low jitter
Mid-size model service	Selected hot layers plus active KV and prompt cache live in HRM	HRM acts as strong accelerator rather than majority capacity
Large model service	Current epoch hot pages, expert subsets, and active KV bands live in HRM	Success depends heavily on residency policy quality and prefetch discipline

15. Bandwidth model

Bandwidth arguments are frequently abused in accelerator marketing, so this section is intentionally disciplined. The point is not to conjure a single heroic number and declare victory. The right question is which bandwidths matter on the useful path and how often each tier must be touched when the residency policy is healthy versus unhealthy.

In the baseline illustrative model, Tier 0 scratch sits in the effectively local bandwidth class, Tier 1 HRM sits in a much higher bandwidth class than off-chip memory because of short wires and banked regional access, Tier 2 HBM delivers multi-terabyte package-local bandwidth, and Tier 3 CXL delivers much lower but still meaningful pooled bandwidth. The architecture wins not by denying HBM bandwidth, but by reducing the fraction of useful token work that must consume it.

One useful normalized way to view the package is to measure the fraction of token-path bytes served by each tier. If the majority of bytes in the steady-state decode path come from HRM, the package behaves like a residency-first processor. If most bytes still come from HBM, the architecture has probably failed to use its expensive SRAM meaningfully.

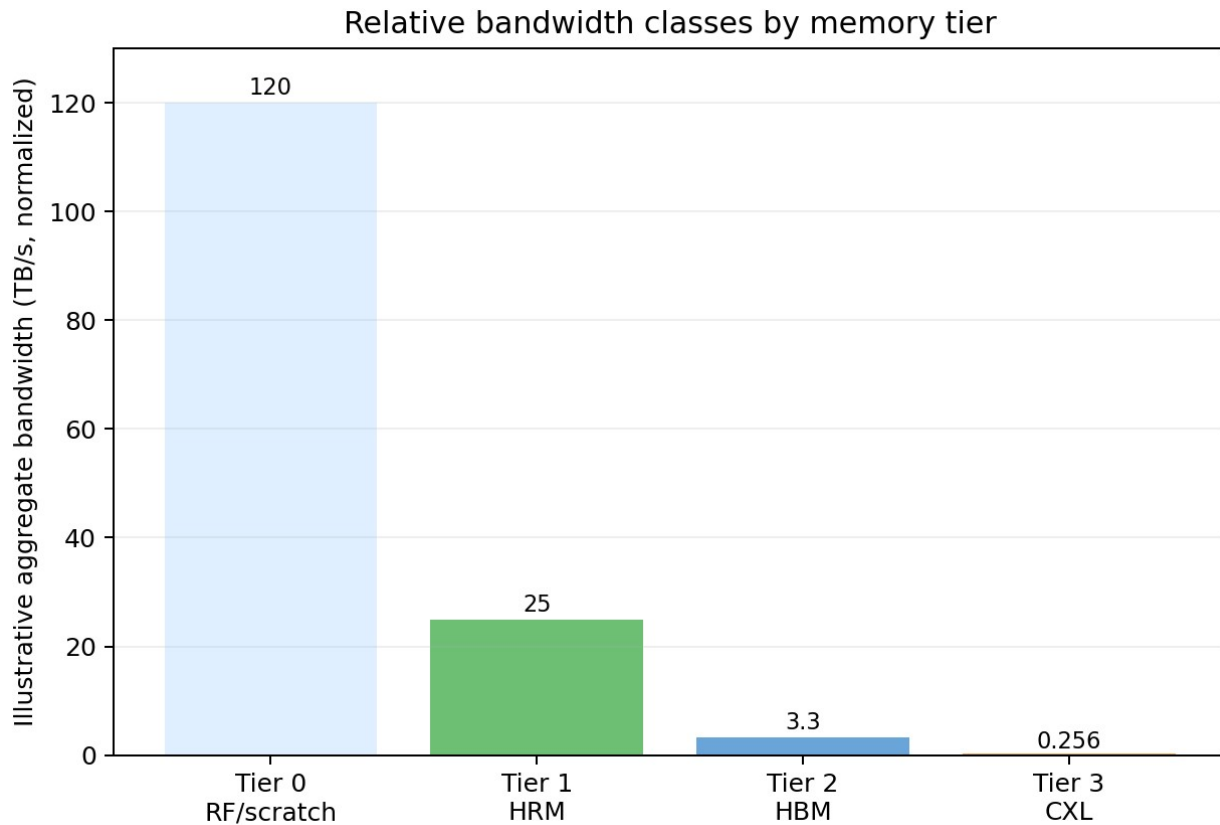


Figure 6. Illustrative bandwidth classes by tier. The point is not the absolute values but the intended separation between local hot-tier movement and colder off-package or off-chiplet movement.

Metric	Healthy residency regime	Unhealthy residency regime
Share of active token bytes served from HRM	High	Low
HBM traffic composition	Mostly controlled staging and misses	Repeated token-path dependence
CXL traffic composition	Warm pooling and prefetch support	Emergency spill / poor-tail latency pressure
Observed token jitter	Lower and more bounded	Higher and more variable

16. Token latency model

For the target deployment class, the useful latency metric is not only first token latency but sustained token latency under real concurrency. Users experience stutter when a supposedly low-latency service repeatedly misses into colder tiers. SRMIC therefore defines success as a higher fraction of token steps completing with HRM-centric rather than HBM-centric paths.

A simple first-order latency decomposition for one token step is: schedule and dispatch time plus any required page-promotion time plus local compute time plus SRMESH movement time plus completion and write-back time. In a strong residency regime, the promotion term is often absent from the critical path because needed pages are already resident or pre-staged. In a weak regime, page promotion enters the path too often and destroys the architecture's purpose.

Therefore the architecture's most important latency lever is not simply faster arithmetic. It is hot-tier hit rate. The control plane, residency controller, and prefetch logic all exist to increase the probability that the next useful block is already in the right HRM region when its TIC asks for it.

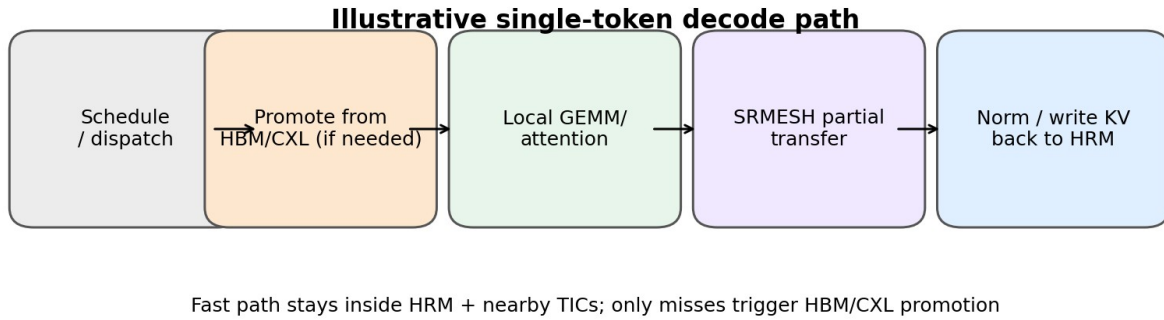


Figure 7. Illustrative single-token pipeline. The package is designed so that the common path should remain within HRM and the local / package residency fabric.

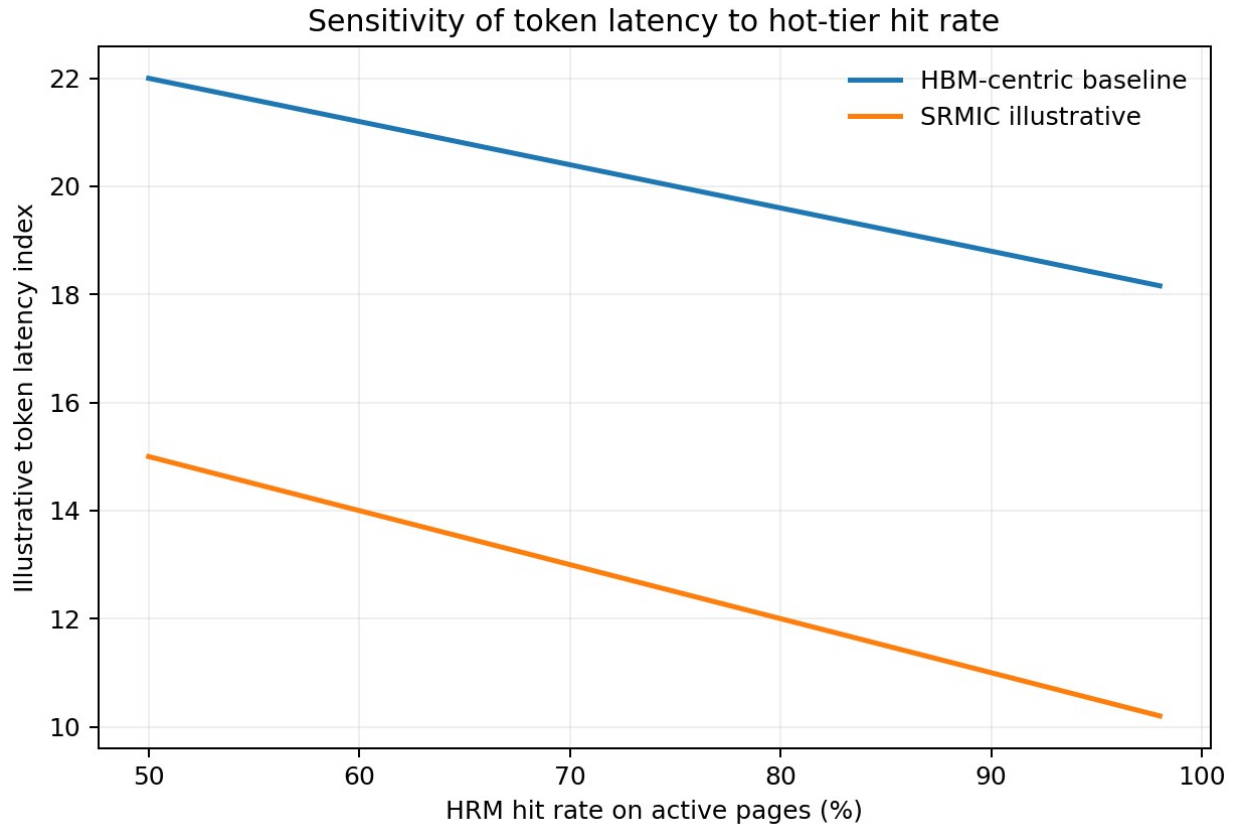


Figure 8. Illustrative sensitivity of token latency to hot-tier hit rate. The figure is conceptual, but it captures the architectural point: hit-rate quality dominates the usefulness of the expensive hot tier.

17. Area allocation model

Area is where architectural ambition becomes concrete. SRAM is not cheap in area, especially at the capacities that make SRMIC interesting. That is precisely why this section matters. If the architecture allocates too much area to CPU-class logic, giant generalized caches, or excessively feature-rich compute tiles, it loses the very memory-dominant posture that is supposed to differentiate it.

The illustrative area split in Figure 9 therefore intentionally puts more than half of the package-level logic real estate into HRM SRAM and its immediate support structures. Compute remains significant, but secondary. The point is not to minimize compute; it is to recognize that this architecture wins only if memory placement is the product.

Because the baseline package uses chiplets, one should think in both per-chiplet and package-level area terms. Each chiplet needs enough local SRAM, enough local compute, and enough local control to be viable even when package-level traffic is not uniform. This again favors the region-first, chiplet-aware decomposition chosen earlier.

SRMIC-P1 package-level area allocation

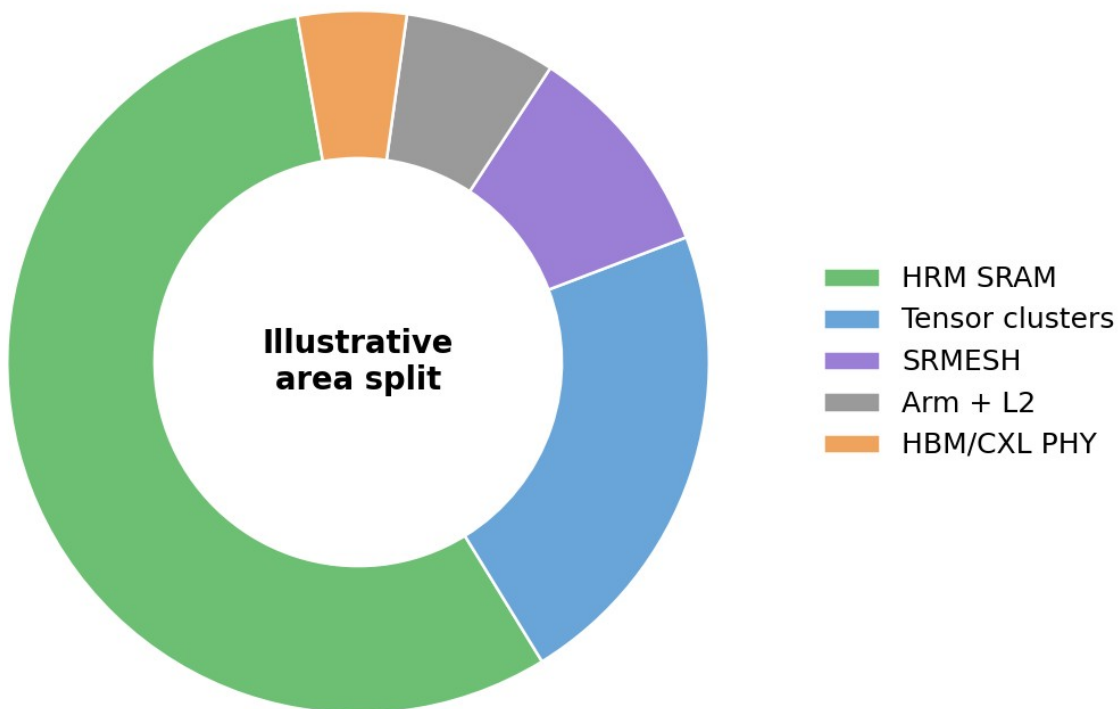


Figure 9. Illustrative package-level area allocation. The architecture remains coherent only if SRAM dominates the die budget rather than being an afterthought.

Important interpretation

The area split shown here is not a foundry-validated floorplan estimate. It is a disciplined reminder of what the architecture implies: if the design drifts away from memory dominance, it drifts away from SRMIC itself.

18. Power and thermal model

Large SRAM is wonderful for latency and painful for leakage. Any honest document must say both sentences. The same is true for wide fabrics and dense compute blocks. SRMIC does not avoid power pressure; it reallocates power from repeated colder-tier movement into a larger on-package hot tier and its supporting fabric.

The baseline package therefore sits in an intentionally high power class. That high class is not a bug in the document; it is an admission that serious data-center inference silicon already lives in a world where power density, package delivery, and thermal extraction are central constraints. The paper therefore assumes strong liquid-assisted or advanced air-cooling deployment rather than pretending the package belongs in a modest edge appliance.

Three thermal concerns deserve special attention. First, HRM leakage can create a large baseline thermal floor. Second, local hotspots may appear where multiple active TICs sit adjacent to the same busy HRM regions. Third, package asymmetry can emerge if HBM traffic is not well balanced. All three need countermeasures: regional gating and drowsy modes for HRM, thermal-aware placement and scheduling, and package-level balancing policies.

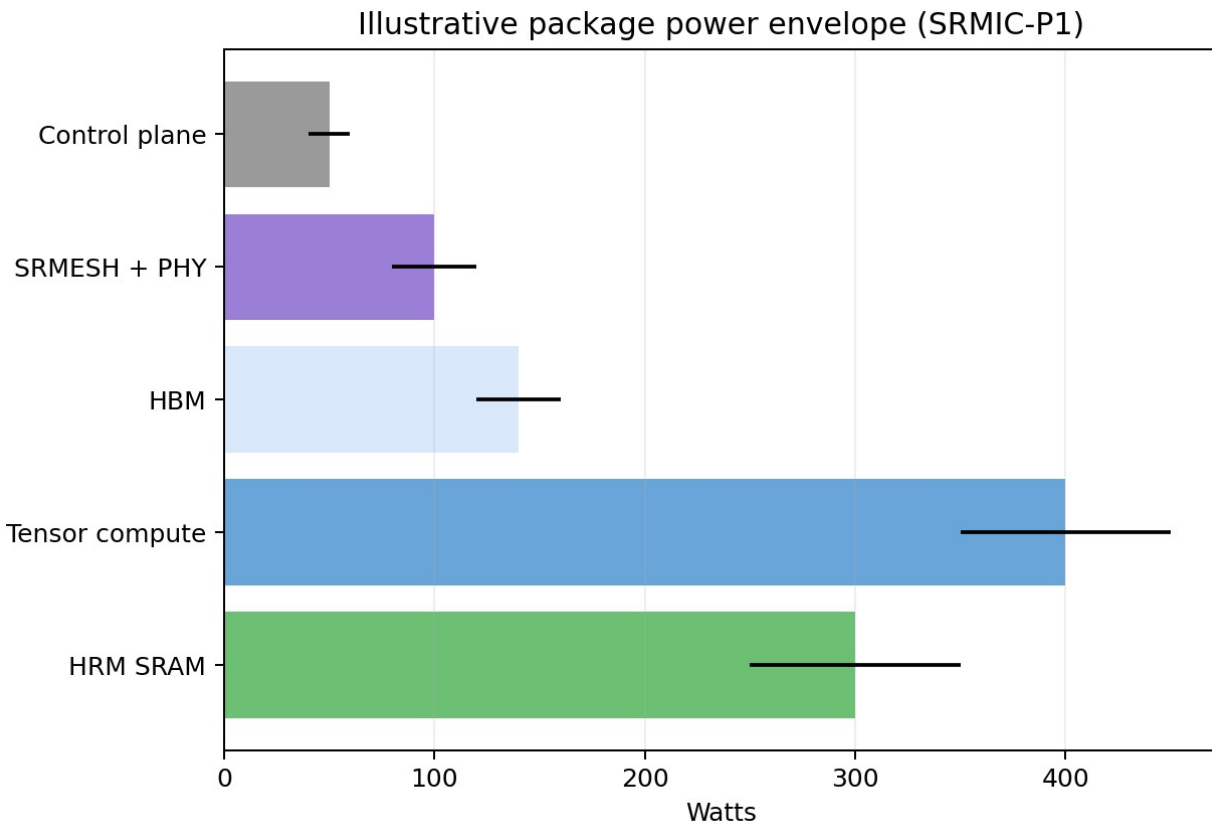


Figure 10. Illustrative package power allocation. Large HRM and substantial tensor throughput imply a premium thermal solution.

- Use fine-grained SRAM bank gating and epoch-based drowsy modes when pages cool off.
- Steer work away from overheated regions if the latency cost is lower than the thermal risk.
- Balance HBM use across stacks and chiplets to avoid persistent edge hotspots.
- Treat thermal telemetry as an input to residency policy, not merely as a passive alarm.

19. Packaging and physical implementation options

There are at least three plausible implementation families for SRMIC, and they differ materially in ambition. The baseline family is a 2.5D multi-chiplet package with HBM around the periphery and no stacked SRAM above logic. This is the most straightforward interpretation of the present document. A second family adds stacked SRAM tiers on top of logic chiplets, increasing hot-tier capacity but also packaging, thermal, and manufacturing complexity. A third, far more radical family approaches wafer-scale or near-wafer-scale ideas. This document deliberately does not choose the radical family as the baseline.

The practical reason to prefer a chiplet baseline is that the architecture already decomposes cleanly by region and chiplet. Chiplets also improve product flexibility. A vendor could imagine smaller and larger packages, or even memory-heavy and compute-heavy bins, without rewriting the entire architecture. That is strategically useful.

The physical implementation still faces demanding questions: power delivery into large SRAM-heavy chiplets, global clocking across a package-scale fabric, link widths among chiplets, and floorplanning for short critical memory paths. None of those questions invalidate the architecture, but they do strongly argue for an implementation roadmap rather than a claim that every feature appears in generation one.

Implementation family	Advantages	Main risks
2.5D multi-chiplet baseline	Most believable near-term path; strong modularity	Chiplet fabric complexity and package routing
3D SRAM over logic roadmap	Higher hot-tier capacity per package	Thermal coupling, cost, and yield complexity
Wafer-scale style extreme	Maximum on-package or on-wafer hot-state vision	Manufacturing and productization risk far beyond baseline

20. Node-level scale-out architecture

One package is useful, but the architecture becomes much more compelling when scaled to a coherent appliance shape. The illustrative node, SRMIC-N1, uses four SRMIC-P1 packages connected through a package-level or board-level fabric and attached to pooled CXL memory. This yields an aggregate of 16 GB HRM, 32 HBM stacks, and 512 tensor inference clusters. The point is not to treat the node as one magical flat memory space; the point is to preserve the same hierarchical residency story at a larger scale.

At node scale, two new concerns appear. The first is placement: which package should own which sessions or hot pages? The second is sharing: when multiple packages could benefit from the same warm pages or prompt artifacts, how much duplication is worthwhile? CXL makes these questions more tractable by providing a pooled warm tier, but the software stack still needs policies for locality, load balance, and service guarantees.

Node scale-out therefore works best when the scheduler is package-aware. A request whose working set is already warm in one package should, when feasible, continue there rather than being bounced around. Cross-package movement should exist, but it should be used for explicit reasons, not as a casual side effect of a generic host scheduler.

Node design question	Preferred answer in SRMIC
Where should an active session live?	Prefer the package already holding the largest fraction of its hot set.
How should warm reusable state be shared?	Prefer pooled CXL plus selective package-local caching over blind duplication everywhere.

When is cross-package movement justified?	When load balance or service-level commitments outweigh locality loss.
What should remain package-local?	The token-critical path for a given active step whenever possible.

21. Competitive positioning

The easiest mistake in positioning SRMIC is to claim victory on the wrong axis. The architecture should not claim that it replaces every HBM-centric GPU, defeats wafer-scale systems on raw on-chip capacity, or renders training accelerators obsolete. That would be unserious. Its strongest claim is narrower and more believable: it targets low-jitter, low-latency inference by making hot-state residency the product.

Against a conventional GPU inference path, SRMIC aims to win when the workload has a meaningful hot set and when the service cares about deterministic decode behavior. Against a wafer-scale SRAM-rich design, SRMIC aims to win on manufacturability, modularity, and ecosystem realism. Against other inference-focused specialized processors, SRMIC aims to differentiate through explicit multi-tier residency semantics and a structured role for CXL pooled warm memory.

This is why language matters. The architecture should be pitched as a residency-first inference fabric, not merely a chip with large cache. Large cache sounds incremental. Residency-first sounds like a different systems philosophy, which is what this architecture is trying to establish.

Comparison axis	Conventional GPU	Wafer-scale SRAM machine	SRMIC intended posture
Primary execution memory	HBM + cache hierarchy	Distributed on-wafer SRAM	Distributed hot SRAM with HBM and CXL behind it
Main strength	Broad programmability and scale	Extreme on-chip locality	Hot-set determinism and modularity
Manufacturing posture	Established high-volume accelerator path	Radical and specialized	Ambitious but more modular package path
Best story for SRMIC	Not peak FLOPS, but lower jitter under good hot-set residency		

22. Risks, open questions, and mitigation plan

Every differentiated architecture lives or dies by its risk discipline. The first risk for SRMIC is economic: SRAM is expensive in area and leakage. If the product does not deliver a service-level advantage that customers feel, then the extra hot-tier investment may not pay back. This is why the architecture must anchor itself to latency-sensitive enterprise workloads rather than vaguely promising everything to everyone.

The second risk is implementation complexity. Wide package fabrics, large SRAM banks, chiplet integration, and rich telemetry all add design burden. A concept can become technically elegant but organizationally unbuildable if too many ambitious features are forced into generation one. That is why the roadmap section matters.

The third risk is software and compiler co-design. A residency-first machine needs a runtime that understands pages, prompts, experts, and heat - not merely tensors. Without a strong software story, the hardware may degrade into a strangely expensive accelerator that still behaves like a cold-memory machine.

Finally, there is a market risk. GPU ecosystems are very strong. Even a technically superior inference path may fail commercially if it demands too much bespoke software work or if it lands before the target customers have the operational maturity to exploit its advantages.

Risk	Why it matters	Mitigation direction
SRAM area and leakage	Could erase product economics	Treat HRM as premium hot tier only; validate hit-rate gains rigorously
Fabric complexity	Can delay or destabilize implementation	Use a hierarchical SRMESH and stage features across generations
Software immaturity	Could waste the hardware advantage	Build a page-aware runtime and strong observability early
Thermal concentration	May reduce sustained performance	Integrate thermal feedback into scheduling and residency policy
Ecosystem inertia	Customers may prefer existing stacks	Position around specific pain points: jitter, appliance deployment, and service quality

23. Suggested development roadmap

A practical roadmap should avoid trying to prove the whole concept in one leap. Generation zero should be a modeling and software phase: page-state metadata design, runtime simulation, hot-set analysis on representative workloads, and bandwidth / latency validation under realistic traces. If the architecture cannot win in a simulator with real traces, it will not win in silicon.

Generation one should focus on the minimum coherent product story: a package with a substantial but not maximal HRM, a restrained SRMESH feature set, strong observability, and a software stack that can actually demonstrate residency-aware gains. Generation two can then expand capacity, multicast sophistication, and possibly stacked SRAM or richer node-level capabilities.

The roadmap should therefore sequence invention and risk carefully: prove the semantics, then the package, then the larger node.

37. Build a trace-driven inference simulator with explicit page-state transitions and region-level placement.
38. Validate that hot-set hit-rate improvements materially reduce jitter on 7B to 34B-class serving workloads.
39. Prototype the runtime and telemetry stack before over-investing in silicon feature breadth.
40. Implement a restrained first SRMESH with essential movement classes and observability.
41. Only after proving generation one, decide whether stacked SRAM or larger pooled warm tiers materially improve the product.

24. Invention / patent framing

On invention grounds, the obvious components of SRMIC are not likely to be the strongest claims by themselves. 'Use SRAM for AI' is not novel in isolation. 'Use Arm as a control plane' is not novel in isolation. 'Use HBM' and 'use CXL' are certainly not novel in isolation. The differentiable claim set emerges from the way the pieces are combined and the explicit semantics assigned to movement and residency.

The strongest conceptual claim family is therefore around residency-managed execution: a package in which on-package SRAM forms a designated hot residency tier, tensor execution clusters are co-located with that tier, and the interconnect understands explicit promote, demote, pin, multicast, and spill semantics across SRAM, HBM, and pooled expansion memory. That is more specific than a broad 'accelerator with multiple memories' claim.

A second claim family is around policy-informed hardware behavior: region-local residency controllers, service-class aware movement on SRMESH, and the coupling of telemetry, heat, and scheduling state into page placement decisions. A third claim family is around node-scale behavior, especially if pooled warm memory and package-aware scheduling are expressed as a coherent serving architecture.

- Candidate claim theme 1: hot-residency SRAM treated as an explicit managed execution tier rather than only as cache.
- Candidate claim theme 2: region-centric compute-memory islands with local page-state engines and package-level residency mesh.
- Candidate claim theme 3: explicit multi-tier page semantics across HRM, HBM, and CXL pooled memory.
- Candidate claim theme 4: package-aware and thermal-aware residency scheduling for low-jitter inference.

25. Appendix A - consolidated assumptions

Assumption class	Chosen value or posture
Process	Advanced leading-edge logic node in the 3 nm class; conceptual study only
Baseline package	4 logic chiplets, 4 GB HRM, 8 HBM stacks, 8 Arm cores, 128 TICs
Memory semantics	Explicit residency-managed Tier 1 hot SRAM, not passive cache only
Primary workload	Decoder-heavy transformer inference and bounded-context enterprise serving
Power class	Approximately 800-1000 W package, premium data-center deployment
Scale-out target	4-package node with pooled CXL warm memory
Document posture	Architecture exploration, not sign-off timing or physical-design closure

26. Appendix B - reference notes

The document is primarily an original design study and therefore relies mostly on internally chosen assumptions rather than on quoted third-party benchmarks. However, several public reference points were used to anchor the discussion: Samsung public materials on HBM4 bandwidth and capacity classes; CXL Consortium material on CXL 3.1 and pooled memory / fabric behavior; Arm public material on CMN and system-level cache / CXL-attached device support; and Cerebras public materials on WSE-3 on-chip SRAM scale and wafer-scale execution posture.

Those sources are used here only as external boundary markers for the design space, not as claims that SRMIC matches or exceeds any one commercial implementation on every axis.

Reference note	Public source family used as a boundary marker
HBM capacity / bandwidth classes	Samsung HBM4 public announcement / product materials
CXL fabric semantics	CXL Consortium white papers and specification release notes
Arm mesh and CXL posture	Arm Neoverse CMN and Neoverse product pages
Wafer-scale SRAM reference point	Cerebras WSE-3 public product and press materials

27. Appendix C - page-state metadata and movement state machine

A residency-first machine needs explicit page-state metadata. Otherwise the hardware cannot distinguish a page that is merely recently touched from a page that is semantically important to the next several decode steps. The minimal metadata structure proposed for SRMIC includes a page identifier, owner or tenancy domain, region placement, quantization form, usage class, age epoch, hotness counter, whether the page is shared, whether the page is currently pinned, whether demotion is legal, and the most recent source tier from which the page was promoted.

The point of the metadata is not to build a perfect oracle. Instead, it is to create enough structure that the runtime, Arm control islands, and region-local residency controllers can cooperate. A page that was recently promoted from HBM because it sits on the active decode path should not be treated identically to a cold page that happened to be touched once. Likewise, a prompt artifact shared across many sessions should not be treated like private per-request scratch.

An illustrative state machine is shown in Figure 11. The machine is intentionally simple: pages can be loaded from colder archival storage, placed into pooled warm memory, staged package-locally in HBM, promoted into HRM, and then optionally pinned during active use. When pressure rises or the page cools, it can be unpinned, demoted, and eventually aged out to a colder tier.

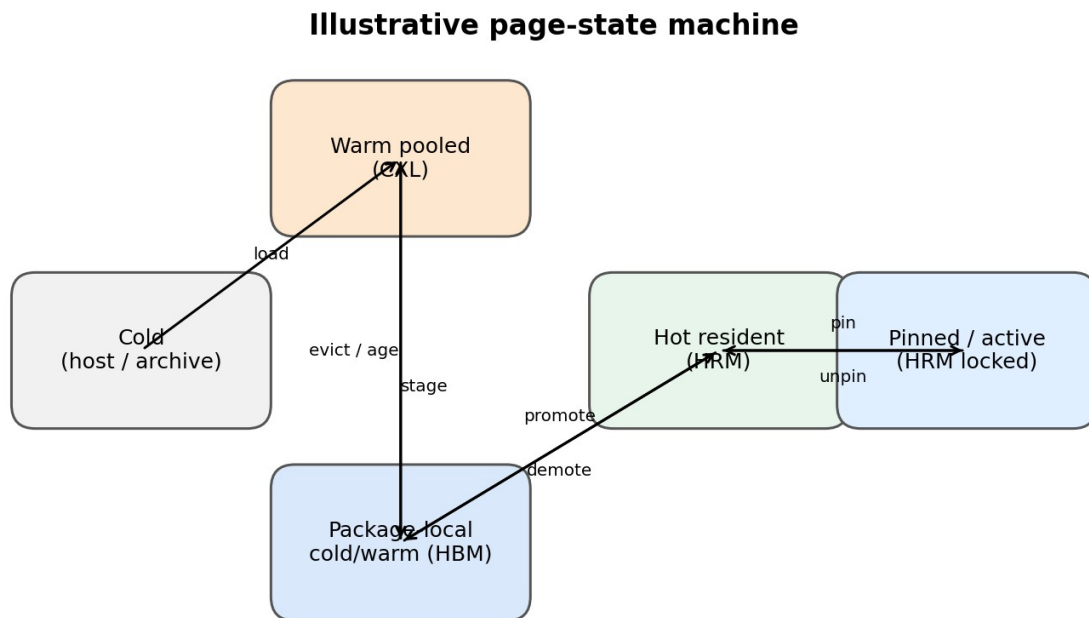


Figure 11. Illustrative page-state machine for SRMIC. The real implementation may use richer substates, but the conceptual lifecycle is *load -> warm -> package-local -> hot -> pinned -> demote*.

Metadata field	Purpose	Why it is useful
Page ID	Unique identity	Allows telemetry, ownership, and movement tracking
Region ID	Current hot placement	Supports local routing and pressure analysis
Tenant / owner	Isolation domain	Prevents unsafe sharing and enables accounting

Format state	Quantized / dequantized / packed	Avoids redundant transforms and clarifies movement costs
Heat / score	Predicted value of staying hot	Drives bounded policy decisions
Pin status	Cannot demote now?	Protects in-flight decode work or service guarantees
Shared flag	Reusable across sessions?	Enables smarter warm-tier pooling and multicast decisions
Last source tier	HBM, CXL, or host	Improves observability and can help policy tuning

28. Appendix D - compiler, runtime, and software contract

SRMIC only works if the software stack participates. The compiler, graph partitioner, and runtime cannot treat the memory hierarchy as an opaque pool and expect the hardware to infer everything. Instead, the system needs a contract. The compiler should be able to identify stable hot blocks, page groupings, and execution boundaries. The runtime should be able to label requests, carry prompt and session metadata, and feed predictions into the residency engine. The hardware should expose descriptors, counters, and bounded policy hooks.

A practical contract has three layers. The first is an offline or compile-time layer that groups model weights and other assets into page units meaningful to SRMIC. The second is a runtime placement layer that chooses package, region, and tier targets. The third is a telemetry loop that feeds miss rates, thermal pressure, and region congestion back into scheduling and placement. If any one of these layers is absent, the architecture degrades toward generic cold-memory behavior.

Figure 12 depicts a useful control-stack view. The package should not require every application to be residency-aware from scratch; that would be a commercial mistake. Instead, a serving runtime or system library should absorb most of the complexity and present manageable interfaces upward.

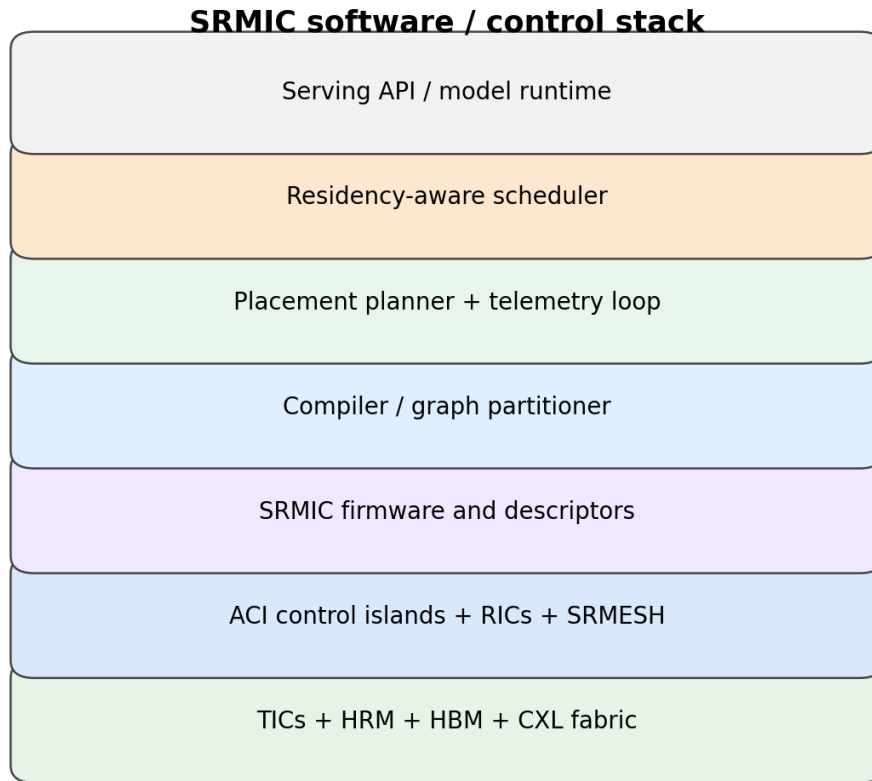


Figure 12. Illustrative software stack. The residency-aware scheduler and placement planner are the key layers that turn the hardware into a useful product.

- The compiler should emit page-group hints, not only kernels.
- The runtime should reason about sessions, prompts, experts, adapters, and tenant constraints.
- The package firmware should expose counters for HRM hit rate, per-region pressure, HBM spill volume, and CXL warm-tier reuse.
- A 'black box' software posture is unlikely to extract enough value from the hot tier.

29. Appendix E - QoS, isolation, and multi-tenant controls

Enterprise inference systems do not run one happy benchmark in isolation. They run mixed tenants, mixed service classes, and unpredictable bursts. SRMIC therefore needs explicit quality-of-service posture. A useful package should support at least three broad request classes: premium low-latency traffic, standard interactive traffic, and best-effort background traffic. The residency engine and scheduler should treat these classes differently.

Isolation matters for two reasons. The obvious one is security and tenancy. The subtler one is performance integrity. If one tenant can thrash the hot tier or the promotion fabric, everyone loses. Therefore the package should track per-tenant budgets or at least per-service-class budgets for HRM footprint, promotion rate, and background prefetch.

In practice, the cleanest model is region-aware quotas plus package-level escape hatches. Premium traffic can reserve bounded HRM and movement headroom. Standard traffic can exploit spare capacity but must yield

under pressure. Best-effort traffic should ride on the leftover budget and should never own the token-critical channels.

Service class	Scheduler bias	Residency policy	Movement priority
Premium low latency	Strong locality preference	More pinning and stricter demotion control	Highest
Standard interactive	Balanced locality and throughput	Moderate pinning, opportunistic prefetch	Medium
Best effort / batchy	Flexible placement	Minimal pinning, yield under pressure	Lowest

Commercial importance

QoS is not polish. It is how the package turns an expensive hot tier into a dependable service-level advantage rather than a benchmark trick.

30. Appendix F - reliability, serviceability, and observability

A residency-first package should be easier to debug than a purely heuristic machine because its movement semantics are explicit. That is an advantage, but only if the package actually exposes the right observability. At minimum, an operator should be able to answer basic questions: what fraction of active token bytes came from HRM, which regions are hottest, which tenants caused most promotions, how often pages were demoted before reuse, and whether thermal throttling altered placement decisions.

Reliability and serviceability also matter. HBM errors, chiplet-link issues, and SRAM bank faults are not theoretical. The package therefore benefits from region-level fencing, graceful degradation modes, and telemetry that can identify bad actors without collapsing the entire node. If one region or link is degraded, the machine should preferably limp in a smaller capacity mode instead of disappearing entirely.

A strong operational story is another place where SRMIC can differentiate. If the appliance can show operators exactly why latency changed - misses, thermal steering, warm-tier saturation, HBM pressure, or noisy neighbors - it becomes much easier to trust the architecture.

- Expose per-region hit rate, promotion volume, demotion regret, thermal load, and queue depth.
- Allow region or bank fencing for degraded-service operation.
- Support trace capture of residency decisions for difficult customer cases.
- Make counters usable by platform teams, not only by chip designers.

31. Appendix G - worked workload scenarios

This appendix reframes the architecture through the lens of three deployment classes. In a compact 7B-class service, the package has a realistic chance of keeping a large fraction of the true active working set in HRM. This is where SRMIC should look strongest. The hot tier can act as a genuine execution substrate rather than as a narrow accelerator of only a few pages.

In a 13B to 34B-class service, the package is still useful, but the story changes. HRM now focuses on selected hot layers, active KV bands, and the most reused prompt artifacts. The runtime and residency engine become more important because the hot tier can no longer behave as a broad reservoir; it must behave as a disciplined residency manager.

In 70B-plus serving, the package should be described honestly as a hot-set accelerator. It may still improve useful latency if the service exhibits strong temporal locality, but the architecture is now much more sensitive to scheduler quality, request mix, and prefetch correctness.

Worked workload scenarios for SRMIC

Service class	Role of HRM	Scheduler pressure
7B class	High useful residency	Simple
13B-34B class	Selective hot residency	Moderate
70B+ class	Epoch hot-set residency	Demanding

Figure 13. Worked workload classes. SRMIC becomes more policy-sensitive as model scale rises and the hot tier becomes a smaller fraction of full model state.

Workload class	Best SRMIC story	Main requirement for success
7B class	High useful residency and low jitter	Keep hot-set model pages and active KV local
13B-34B class	Strong hot-set acceleration	Good policy discipline and predictive staging
70B+ class	Selective hot-path acceleration	Excellent placement, prefetch, and request locality

32. Appendix H - sensitivity and what must be true for SRMIC to win

A disciplined architecture memo should state the conditions under which it loses. SRMIC loses if the workload does not have a meaningful hot set, if the software stack cannot predict or preserve locality, if package thermals force constant steering away from the ideal regions, or if CXL and HBM are used so frequently that the fast path becomes rare. In those cases the expensive SRAM turns into a weakly utilized tax.

Conversely, SRMIC wins when the next-use probability of a bounded set of pages is high enough that keeping them in HRM changes a large share of token steps from cold-tier-dependent to hot-tier-resident. That is why hit-rate quality is the central sensitivity parameter. The architecture is best understood as a curve, not a binary statement.

The practical use of this appendix is to force future review meetings to ask the right questions. What is the measured hot-set size? How stable is it over a session? What is the regret of each demotion policy? How much does tenant mix erode locality? If those questions are not answered, the architecture remains rhetoric.

Condition	If true, SRMIC gets stronger	If false, SRMIC weakens
Hot-set locality	High temporal reuse in bounded windows	Large random working set with weak repetition
Runtime quality	Can identify hot pages and stage them early	Sees only tensors, not page-value semantics
Thermal envelope	Can sustain region-local work without frequent steering	Constant hotspot avoidance disrupts locality
Request affinity	Sessions can stay where their hot pages already live	Scheduler thrashes requests across packages

33. Appendix I - product stack and packaging roadmap

Different customers may want different product shapes. A smaller SKU could offer less HRM, fewer TICs, and fewer HBM stacks while preserving the same software and residency model. A larger SKU could scale chiplet count or package richness. This flexibility is one of the reasons to prefer a modular package strategy over a monolithic extreme.

The roadmap figure below suggests a sensible sequencing: prove the trace model, then the single package, then richer mesh features, then higher-capacity memory options, then the appliance node. This is not merely prudent engineering; it is good product strategy. Each generation should have a distinct story rather than a vague promise that everything gets bigger.

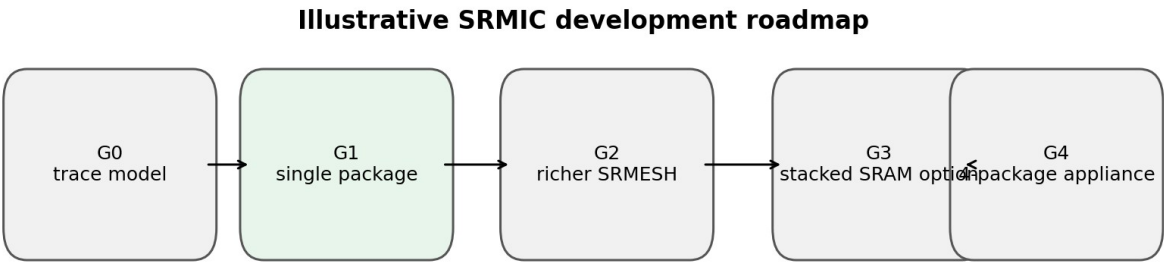


Figure 14. Illustrative development roadmap. A credible residency-first product should prove the software and movement semantics before overreaching on package heroics.

Possible SKU	Shape
SRMIC-S	Smaller package, lower HRM and HBM count, same core residency semantics
SRMIC-P1	Baseline premium package described in this document
SRMIC-P2	Higher-capacity package with richer hot tier or packaging advances
SRMIC-N1 appliance	Four-package node with pooled warm memory and software stack tuned for service deployment

34. Appendix J - due-diligence questions for the next review

42. What is the measured hot-set size distribution for the target serving workloads, and how stable is it over time?
43. What fraction of token-path bytes can plausibly be served from HRM under realistic traces rather than idealized traces?
44. How much of the package power budget is leakage versus dynamic traffic, and how does that change with duty cycle?
45. At what point does CXL pooled warm memory materially improve service economics instead of merely increasing complexity?
46. What is the minimum software stack required to demonstrate a convincing generation-one win against a credible baseline?
47. What are the top three physical-design unknowns for the 2.5D multi-chiplet baseline, and how can they be de-risked quickly?
48. What exact observability will operators need to trust the residency engine in production?
49. Which parts of the concept are invention-worthy and which parts should be treated as commodity implementation choices?

35. Appendix K - glossary and design shorthand

This final appendix collects the shorthand used throughout the document so that design reviews can move quickly without constantly re-establishing terminology. In many hardware projects, confusion over terms creates avoidable disagreement. A short glossary therefore improves both communication quality and review efficiency.

Term	Meaning in this document
ACI	Arm Control Island; the control-plane CPU cluster responsible for scheduling, telemetry, and protection services
HRM	Hot Residency Memory; the on-package SRAM tier intended to hold active pages for the fast path
TIC	Tensor Inference Cluster; the local execution block adjacent to HRM within a region
RIC	Residency Interconnect Controller; region-local page-state and movement control logic
SRMESH	Package-level residency-aware movement fabric tying regions together
Warm tier	Memory outside the package hot path but still worth retaining, typically via CXL pooling
Cold tier	Package-local HBM tier for colder model pages, overflow state, and staging
Demotion regret	A useful metric: how often a page demoted from HRM is needed again too soon
Hot-set hit rate	Fraction of active token-path bytes served from HRM rather than colder tiers

A good rule for future review sessions is simple: if a statement about SRMIC does not clarify which tier it is talking about, the statement is probably too vague to be useful.

36. Appendix L - design review checklist and FAQ

This appendix is deliberately operational. The goal is to make the next design review more disciplined by turning the document into a checklist. A strong concept can still fail if review meetings stay vague. The questions below are the ones that should be answered before the concept advances from architecture memo to funded engineering plan.

Checklist topic	Question that must be answered
Trace realism	Do we have representative traces from the exact serving regime we want to win?
Hot-set sizing	Can we quantify hot-set bytes by percentile, not only by anecdote?
Policy quality	Do promote, pin, and demote policies beat a simpler baseline in simulation?
Thermal sustainability	Can the preferred locality plan survive realistic sustained thermal conditions?
Software burden	What minimum runtime and compiler changes are required for generation one?
Commercial story	Which customer pain point improves enough to justify architectural novelty?
Fallback behavior	If locality is poor, does the product fail gracefully or collapse into a weak expensive baseline?

Frequently asked questions are equally useful because they expose where the concept may be over- or under-claimed.

- Does SRMIC replace HBM? No. HBM remains essential as the package-local cold tier.
- Is SRMIC only a cache idea? No. The key claim is explicit residency semantics, not merely a larger LLC.
- Does the package need Arm specifically? Arm is a strong control-plane choice, but the bigger point is that the CPU must remain a control instrument rather than the center of tensor traffic.
- Can SRMIC beat wafer-scale systems on raw on-package memory? Not necessarily; that is not the intended primary claim.
- What must be true for SRMIC to be commercially compelling? The target workloads must care about jitter and must exhibit enough hot-set locality to justify the premium hot tier.